Documentation

Overview

This project is a TCP-based client-server application written in Python. The project makes use of the socket library and implements the stop and wait protocol to establish a reliable connection.

Some core features:

- User login authentication
- Sending text messages
- Uploading files
- Graceful client disconnects

The system consists of:

- `server.py` — handles incoming client connections and processes commands
- `client.py` — command-line client used to communicate with the server

---

Project Structure

```
.
├── client.py
├── server.py
├── users.txt
└── received_files/
```

- `users.txt` stores valid usernames
- `received_files/` stores uploaded files received by the server

Requirements

- Python 3.x
- Standard Python libraries only:
  - `socket`
  - `os`
  - `sys`

No external packages are required.

Starting the Program

Starting the Server

Open your terminal, and paste in this command:

```
python server.py
```

The command will:

- Load valid usernames from `users.txt`
- Start listening on port `9372`
- Accept one client connection at a time

Starting the Client

Open a *second* terminal, and paste in this command:

```
python client.py
```

You can optionally specify a custom host and port. Host defaults to `127.0.0.1`, and port defaults to `9372`:

```
python client.py <host> <port>
```

Example:

```
python client.py
python client.py 127.1.2.3
python client.py 127.1.2.3 3000
```

The terminal will start the custom CLI for this applcation after this command is run.

Supported Commands

There are four supported commands:

Login

Sends a login request to the server. If the user is found in `users.txt`, the user is authenticated. This command must be run to be able to use the other commands.

Syntax

```
LOGIN <username>
```

Example Usage

```
LOGIN Alice
```

Example Responses

```
200 OK Welcome, alice
401 UNAUTHORIZED Invalid username
400 ERROR Missing username
```

MSG

Send a text message request to the server. The user must be logged in first to use this command.

Syntax

```
MSG <message>
```

Example Usage

```
MSG Hello server
```

Example Responses

```
200 OK Message received: Hello server
403 FORBIDDEN Please login first
400 ERROR Empty message
```

FILE

Upload a file to the server.

Syntax

```
FILE <filepath>
```

Example Usage

```
FILE notes.txt
```

Example Responses

```
200 OK File 'notes.txt' received (1024 bytes)
400 ERROR Missing filename
400 ERROR Invalid file size
403 FORBIDDEN Please login first
```

Implementation

The client sends:

1. `FILE <filename>`
2. File size
3. Raw file bytes

The server:

- Validates the file size
- Saves the file into `received_files/`
- Prevents directory traversal using `os.path.basename()`

Limitations

- Supports a maximum file size of **100 MB**

QUIT

Disconnect from the server. Must be logged in with an established connection to use this command.

Syntax

```
QUIT
```

Response

```
200 OK Goodbye
```

Authentication

Valid usernames are stored in `users.txt`.
Example:

```
Alice
Bob
Charlie
```

Only usernames listed in this file are allowed to log in.

Error Handling

The system handles several error cases:

- Empty commands
- Invalid usernames
- Unauthorized actions
- Missing files
- Invalid file sizes
- Unexpected client disconnects

The client also handles:

- Connection failures
- Keyboard interrupts (`Ctrl+C`)
- Server disconnects

Security Notes

Implemented protections include:

- Filename sanitization using `os.path.basename()`
- File size limit (100 MB)
- Authentication before messaging or file upload

Limitations:

- No password authentication
- No encryption (plain TCP)
- Single-threaded server (one client at a time)

Example Session

Client Terminal

```
> LOGIN Alice
[Server] 200 OK Welcome, Alice

> MSG Hello
[Server] 200 OK Message received: Hello

> FILE test.txt
[Server] 200 OK File 'test.txt' received (45 bytes)

> QUIT
[Server] 200 OK Goodbye
```

Server Terminal

```
[AUTH] User logged in: alice
[MSG] alice: Hello
[FILE] Received 'test.txt' (45 bytes) from alice
```